



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251221
Nama Lengkap	RIZKY FEBRIANTO KRISTIANSYAH
Minggu ke / Materi	13 / Tipe Data Tuples

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: Materi

Tuple Immutable

Tuple merupakan salah satu tipe data dalam Python yang perilakunya menyerupai list. Nilai yang disimpan dalam tuple dapat berupa apa pun dan diberikan indeks berupa bilangan bulat (integer). Perbedaan utama antara tuple dan list terletak pada sifatnya: tuple bersifat immutable, artinya anggota yang sudah ada di dalam tuple tidak dapat diubah setelah tuple tersebut dibuat. Selain itu, tuple juga bersifat hashable sehingga dapat digunakan sebagai key dalam dictionary Python, sedangkan list tidak bisa digunakan sebagai key dictionary karena bersifat mutable.

Sintaks penulisan tuple dapat dilakukan dengan dua cara: menggunakan tanda kurung atau tanpa tanda kurung, cukup dengan memisahkan nilai-nilai menggunakan koma. Untuk tuple yang hanya berisi satu elemen, wajib ditambahkan tanda koma di belakang elemen tersebut agar Python mengenalinya sebagai tuple dan bukan sebagai string biasa. Selain itu, tuple juga dapat dibuat menggunakan fungsi bawaan tuple() yang apabila dipanggil tanpa argumen akan menghasilkan tuple kosong, dan apabila diberi argumen berupa string atau list maka akan menghasilkan tuple berisi elemen-elemen dari argumen tersebut.

```
>>> t = ('a', 'b', 'c', 'd', 'e')
>>> print(t[0])
a
>>> print(t[1:3])
('b', 'c')
>>> t[0] = 'A'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
>>> t = ('A',) + t[1:]
>>> print(t)
('A', 'b', 'c', 'd', 'e')
```

Gambar 1.1: Contoh dasar penggunaan tuple dan sifat immutable

Meskipun elemen dalam tuple tidak dapat diubah langsung, tuple tetap bisa 'diganti' dengan cara membuat tuple baru menggunakan penggabungan (concatenation). Misalnya untuk mengganti elemen pertama, dapat dilakukan dengan membuat tuple baru yang terdiri dari elemen pengganti ditambah dengan slice tuple lama mulai dari indeks ke-1. Sebagian besar

operator yang bekerja pada list juga berlaku pada tuple, termasuk penggunaan tanda kurung kotak [] untuk mengakses elemen berdasarkan indeks dan slicing untuk mengambil rentang elemen.

Membandingkan Tuple

Operator perbandingan (comparison) dapat bekerja pada tuple maupun tipe data sekuensial lainnya seperti list, dictionary, dan set. Cara kerjanya adalah dengan membandingkan elemen pertama dari masing-masing sekuensial. Apabila elemen pertama sama, perbandingan akan dilanjutkan ke elemen berikutnya, dan seterusnya hingga ditemukan perbedaan. Prinsip ini dimanfaatkan dalam fungsi sort() Python untuk mengurutkan list yang berisi tuple.

```
1 kalimat = 'but soft what light in yonder window breaks'
2 dafkata = kalimat.split()
3 t = list()
4 for kata in dafkata:
5     t.append((len(kata), kata))
6
7 t.sort(reverse=True)
8
9 urutan = list()
10 for length, kata in t:
11     urutan.append(kata)
12
13 print(urutan)
```

Fitur perbandingan tuple ini dikenal dengan pola DSU (Decorate, Sort, Undecorate). Tahap Decorate adalah membangun list tuple dengan satu atau lebih key pengurutan sebelum elemen dari sekuensial. Tahap Sort adalah mengurutkan list tuple menggunakan fungsi sort(). Tahap Undecorate adalah mengekstrak elemen yang telah diurutkan. Sebagai contoh, untuk mengurutkan kata-kata dalam sebuah kalimat dari yang paling panjang ke yang paling pendek, dapat dibuat list tuple berisi pasangan (panjang_kata, kata) lalu diurutkan secara descending.

Gambar 1.2: Source code pengurutan kata menggunakan pola DSU

Penugasan Tuple

Salah satu fitur unik dari Python adalah kemampuannya untuk menempatkan tuple di sisi kiri dari statement penugasan (assignment). Hal ini memungkinkan penetapan nilai ke lebih dari satu variabel sekaligus dalam satu baris pernyataan. Misalnya, apabila terdapat sebuah list

dengan dua elemen, kedua elemen tersebut dapat langsung ditetapkan ke dua variabel berbeda hanya dengan satu statement. Python akan menterjemahkan sintaks ini dengan mengambil elemen pertama untuk variabel pertama dan elemen kedua untuk variabel kedua.

Penugasan tuple juga memungkinkan pertukaran nilai dua variabel dalam satu statement tanpa memerlukan variabel sementara (temporary variable), yaitu dengan penulisan `a, b = b, a`. Seluruh ekspresi di sisi kanan dievaluasi terlebih dahulu sebelum penugasan dilakukan. Yang perlu diperhatikan adalah jumlah variabel di sisi kiri harus sama dengan jumlah nilai di sisi kanan, jika tidak Python akan menghasilkan `ValueError`. Penugasan tuple juga sering digunakan untuk memecah hasil dari fungsi yang mengembalikan beberapa nilai sekaligus, seperti memisahkan alamat email menjadi username dan domain menggunakan `split('@')`.

```
>>> m = ['have', 'fun']
>>> x, y = m
>>> x
'have'
>>> y
'fun'
>>> x, y = y, x
>>> x
'fun'
>>> y
'have'
>>>
>>> email = 'didanendya@ti.ukdw.ac.id'
>>> username, domain = email.split('@')
>>> print(username)
didanendya
>>> print(domain)
ti.ukdw.ac.id
```

Gambar 1.3: Contoh penugasan tuple dan pertukaran value variabel

Dictionary dan Tuple

Dictionary memiliki method bernama `items()` yang berfungsi untuk mengembalikan list dari tuple, di mana setiap tuple-nya merupakan pasangan kunci-nilai (key-value pair). Karena hasilnya berupa list dari tuple, maka dapat dilakukan pengurutan menggunakan `sort()` untuk menampilkan isi dictionary secara teratur berdasarkan kuncinya. Kombinasi antara method `items()`, penugasan tuple, dan perulangan `for` dapat menghasilkan kode yang lebih efisien untuk mengakses kunci dan nilai dari dictionary secara bersamaan dalam satu loop.

Untuk menampilkan isi dictionary yang diurutkan berdasarkan nilainya (bukan kunci), dapat dilakukan dengan cara membuat list tuple di mana posisi elemen dibalik menjadi (value,

key). Setelah list tersebut diurutkan menggunakan `sort()`, elemen pertama setiap tuple (value) yang akan menjadi dasar pengurutan. Apabila terdapat dua tuple dengan nilai yang sama, pengurutan akan dilanjutkan ke elemen kedua yaitu key, sehingga tuple dengan nilai sama akan diurutkan secara alfabetis.

```
>>> d = {'a':10, 'b':1, 'c':22}
>>> t = list(d.items())
>>> t.sort()
>>> t
[('a', 10), ('b', 1), ('c', 22)]
```

Gambar 1.4: Contoh pengurutan dictionary menggunakan tuple

Multipenugasan dengan Dictionary

Kombinasi antara method `items()`, penugasan tuple, dan perulangan `for` dapat dimanfaatkan untuk melakukan multipenugasan sekaligus dalam satu loop. Karena method `items()` mengembalikan list dari tuple pasangan (key, value), maka pada setiap iterasi perulangan `for` kedua elemen tersebut dapat langsung di-unpack menjadi dua variabel terpisah secara bersamaan. Pendekatan ini membuat kode menjadi lebih ringkas dibandingkan harus mengakses kunci dan nilai secara terpisah menggunakan operator indeks.

Apabila ingin menampilkan isi dictionary yang diurutkan berdasarkan nilainya, langkah yang dilakukan adalah pertama membuat list tuple baru dengan posisi elemen dibalik menjadi (value, key) agar pengurutan dilakukan berdasarkan value. Setelah list tersebut diurutkan secara descending menggunakan `sort(reverse=True)`, hasilnya dicetak menggunakan multipenugasan tuple dalam perulangan `for`. Jika terdapat dua tuple dengan nilai yang sama, Python secara otomatis akan melanjutkan perbandingan ke elemen kedua yaitu key, sehingga data dengan nilai sama akan diurutkan secara alfabetis berdasarkan kuncinya.

```
>>> d = {'a':10, 'b':1, 'c':22}
>>> l = list()
>>> for key, val in d.items() :
...     l.append( (val, key) ) ...
>>> l
[(10, 'a'), (22, 'c'), (1, 'b')]
>>> l.sort(reverse=True)
>>> l
[(22, 'c'), (10, 'a'), (1, 'b')]
```

Kata yang Sering Muncul

Pada bagian ini ditunjukkan bagaimana tuple dan dictionary dapat dikombinasikan untuk melakukan analisis teks yang lebih kompleks, yaitu menampilkan 10 kata yang paling sering muncul dalam sebuah file teks. Bagian pertama program membaca file dan membangun dictionary yang memetakan setiap kata ke jumlah kemunculannya. Sebelum kata dimasukkan ke dictionary, setiap baris terlebih dahulu dihapus tanda bacanya menggunakan `translate()` dan dikonversi ke huruf kecil menggunakan `lower()` agar kata yang sama dalam variasi penulisan dihitung sebagai satu kata.

Bagian kedua program melakukan pengurutan berdasarkan frekuensi. Dari dictionary yang sudah terbentuk, dibuat list tuple dengan posisi (value, key) agar pengurutan dilakukan berdasarkan jumlah kemunculan. List tersebut kemudian diurutkan secara descending menggunakan `sort(reverse=True)`. Apabila terdapat dua kata dengan frekuensi yang sama, Python akan melanjutkan perbandingan ke elemen kedua yaitu nama katanya sehingga diurutkan secara alfabetis. Pada bagian akhir, dilakukan iterasi dengan multipenugasan tuple untuk mencetak 10 kata teratas menggunakan slicing `lst[:10]`.

```
import string
fhand = open('romeo-full.txt')
counts = dict()
for line in fhand:
    line = line.translate(str.maketrans('', '', string.punctuation))
    line = line.lower()
    words = line.split()
    for word in words:
        if word not in counts:
            counts[word] = 1
        else:
            counts[word] += 1

lst = list()
for key, val in list(counts.items()):
    lst.append((val, key))

lst.sort(reverse=True)

for key, val in lst[:10]:
    print(key, val)
```

Gambar 1.6: Contoh source code mencari kata yang sering muncul

Tuple sebagai Kunci Dictionary

Karena tuple bersifat hashable, tuple dapat digunakan sebagai kunci (key) dalam dictionary, sedangkan list tidak bisa karena bersifat mutable. Kemampuan ini berguna untuk membuat composite key, yaitu kunci yang terdiri dari lebih dari satu nilai sekaligus. Sebagai contoh, untuk membuat direktori telepon yang memetakan dari pasangan nama belakang dan

nama depan ke nomor telepon, tuple (last-name, first-name) dapat digunakan langsung sebagai kunci dictionary. Saat melakukan iterasi pada dictionary dengan tuple sebagai kunci, elemen-elemen dari tuple kunci tersebut dapat langsung di-unpack menjadi variabel-variabel terpisah menggunakan penugasan tuple dalam perulangan for.

```
directory = dict()
last = 'nendya'
first = 'dida'
number = '088112266'
directory[last, first] = number
for last, first in directory:
    print(first, last, directory[last, first])
```

Gambar 1.7: Contoh tuple sebagai kunci dictionary

BAGIAN 2: LATIHAN MANDIRI

Link Github repository: https://github.com/ifgodwillsit/71251221_febrian

Latihan 13.1

```
tA = eval(input())
hasil = len(set(tA)) == 1
print(hasil)
```

Gambar 2.1: Source code Latihan 13.1

Konsep latihan ini adalah mengecek apakah semua anggota tuple bernilai sama. `eval(input())` memungkinkan user mengetik langsung dalam format tuple seperti (90, 90, 90, 90) dan Python langsung mengenalinya sebagai tuple tanpa perlu konversi tambahan. Pengecekan dilakukan dengan mengonversi tuple ke `set()` sehingga duplikat tereliminasi — jika `len(set(tA)) == 1` maka semua elemen sama dan hasilnya `True`, sebaliknya `False`.

```
(90, 90, 90, 90)
True
```

Gambar 2.2: Output Latihan 13.1

Latihan 13.2

```
data = eval(input("Masukkan tuple (nama, nim, alamat): "))
nama, nim, alamat = data

print("Data :", data)
print("NIM :", nim)
print("NAMA :", nama)
print("ALAMAT:", alamat)

nim_tuple = tuple(nim)
print("NIM      :", nim_tuple)

nama_depan = tuple(nama.split()[0])
print("NAMA DEPAN :", nama_depan)

nama_terbalik = tuple(nama.split()[::-1])
print("NAMA TERBALIK:", nama_terbalik)
```

Gambar 2.3: Source code Latihan 13.2

Konsep latihan ini adalah mengolah data diri menggunakan berbagai operasi tuple. User menginput langsung satu tuple berisi tiga elemen (nama, nim, alamat) menggunakan `eval(input())`, lalu ketiga elemen di-unpack ke variabel masing-masing dalam satu baris penugasan tuple. NIM dikonversi ke tuple karakter dengan `tuple(nim)`, nama depan diambil

dengan `split()[0]` untuk mengambil kata pertama, dan nama terbalik didapat dengan `split()[::-1]` yang membalik urutan kata sebelum dikonversi ke tuple.

```
Masukkan tuple (nama, nim, alamat): ("Rizky Febrianto Kristiansyah", "71251221", "Sapen Gondokusuman")
Data : ('Rizky Febrianto Kristiansyah', '71251221', 'Sapen Gondokusuman')
NIM : 71251221
NAMA : Rizky Febrianto Kristiansyah
ALAMAT: Sapen Gondokusuman
NIM : ('7', '1', '2', '5', '1', '2', '2', '1')
NAMA DEPAN : ('R', 'i', 'z', 'k', 'y')
NAMA TERBALIK: ('Kristiansyah', 'Febrianto', 'Rizky')
```

Gambar 2.4: Output Latihan 13.2

Latihan 13.3

```
fname = input('Enter a file name: ')
try:
    fhand = open(fname)
except FileNotFoundError:
    print('File tidak bisa dibuka:', fname)
    quit()

counts = dict()
for line in fhand:
    if not line.startswith('From '):
        continue
    words = line.split()
    jam = words[5].split(':')[0]
    counts[jam] = counts.get(jam, 0) + 1

lst = list()
for key, val in counts.items():
    lst.append((key, val))

lst.sort()

for jam, jumlah in lst:
    print(jam, jumlah)
```

Gambar 2.5: Source code Latihan 13.3

Konsep dari latihan ini adalah menghitung distribusi jam dalam satu hari di mana pesan email diterima menggunakan file `mbox-short.txt` yang diinput langsung oleh user. File `mbox-short.txt` merupakan format standar yang berisi multiple mail message, di mana setiap pesan dipisahkan oleh baris yang diawali 'From ' (dengan spasi). Setiap baris pemisah tersebut memiliki format: `From <email> <hari> <bulan> <tanggal> <waktu> <tahun>`, contohnya `From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008`. Baris tersebut dipecah menggunakan `split()` yang secara otomatis menangani spasi ganda, sehingga kata keenam (index 5) selalu berupa waktu dalam format `HH:MM:SS`. Jam diambil menggunakan `split(':')[0]` untuk

mengambil dua digit pertama saja, kemudian digunakan sebagai kunci dalam dictionary counts yang nilainya bertambah setiap kali jam yang sama ditemukan. Setelah semua baris diproses, dictionary dikonversi menjadi list of tuple berisi pasangan (jam, jumlah) lalu diurutkan secara ascending menggunakan sort() agar ditampilkan dari jam terkecil ke terbesar.

```
Enter a file name: mbox-short.txt
04 3
06 1
07 1
09 2
10 3
11 6
14 1
15 2
16 4
17 2
18 1
19 1
```

Gambar 2.6: Output Latihan